

Kubernetes Architecture & Distributed Orchestration

A Structured Technical Reference on Control Plane Mechanics, Worker Node Daemons, Declarative Reconciliation, and Workload Primitives

Doc ID: GL-CORPUS-02

Domain: Cloud Orchestration

Topic: Kubernetes Internals

Target: Graph RAG System

1. Prerequisites & Foundational Knowledge

To understand Kubernetes architecture, an engineer must possess a solid grounding in container fundamentals and distributed systems. Kubernetes does not replace container runtimes; rather, it coordinates, manages, and automates containers across large clusters of physical or virtual machines.

The reader should be familiar with the following foundational concepts before proceeding:

- **Container Runtime Mechanics:** Familiarity with Linux Namespaces, Control Groups (cgroups v2), layered images (OverlayFS), and low-level runtime engines (containerd, runc), as detailed in Document GL-CORPUS-01.
- **Client-Server & RESTful APIs:** Understanding HTTP/HTTPS, JSON schemas, gRPC remote procedure calls, and Transport Layer Security (TLS) authentication using public-key certificates.
- **Distributed Consensus Concepts:** Basic familiarity with leader election, quorum, and state replication across multi-node distributed clusters (such as the Raft consensus algorithm).
- **Cloud Networking Basics:** Familiarity with IP address allocation, subnetting, Domain Name System (DNS) resolution, virtual routing, and OSI Layer 4/7 load balancing.
- **Cloud Compute Infrastructure:** Understanding virtual machine fleets such as Amazon EC2 instances and managed cloud clusters such as Amazon Elastic Kubernetes Service (Amazon EKS).

2. Core Kubernetes Concepts & Terminology

This section defines the fundamental entities of Kubernetes. Each concept represents an essential node in cloud-native container orchestration.

Concept: Kubernetes (K8s)

- Beginner-Friendly Definition:** Kubernetes is an open-source container orchestration engine that automates the deployment, scaling, networking, and lifecycle management of containerized applications across a cluster of machines.
- Why It Exists:** Managing individual containers manually on individual virtual machines is impractical at cloud scale. When a server fails, containers crash, or traffic spikes occur, Kubernetes automatically restarts containers, redistributes workloads, and scales capacity.
- How It Works:** Kubernetes operates on a declarative desired-state model. Users declare how their systems should look in YAML configuration files, and continuous background control loops actively reconcile the actual cluster state with this desired state.
- Real-World Cloud Example:** Amazon EKS provisions and operates managed Kubernetes control planes across multiple AWS Availability Zones.
- Related Concepts:** Control Plane, Worker Node, Pod, Deployment, Kubelet, etcd.

Concept: Pod

Beginner-Friendly Definition: A Pod is the smallest, most basic deployable computing unit in Kubernetes. It represents a single instance of an application workload, consisting of one or more tightly coupled containers that share networking, storage, and execution specifications.

Why It Exists: Containers often need to cooperate closely (such as an application container and a logging sidecar). The Pod abstraction groups these containers together onto the same host machine with a shared network namespace and IP address.

How It Works: When Kubernetes schedules a Pod, the container runtime creates a shared Linux Network Namespace and shared IPC Namespace for all containers within that Pod. All containers in the Pod share the IP address `127.0.0.1` locally.

Real-World Cloud Example: An Amazon EKS Pod running a web server container alongside an AWS CloudWatch observability sidecar container.

Related Concepts: Container, Network Namespace, Sidecar Pattern, Deployment, Kubelet.

Concept: Declarative Desired State & Reconciliation

Beginner-Friendly Definition: Declarative management is an architectural approach where the user specifies the desired end outcome (e.g., "maintain 5 replicas of the backend API"), and the system automatically determines and executes the steps needed to reach and preserve that outcome.

Why It Exists: Imperative scripting ("create server 1, create server 2") fails when machines crash unpredictably. Declarative systems continuously repair deviations caused by hardware failures or network partitions.

How It Works: Kubernetes controllers run an infinite loop: *Observe Actual State* → *Compare with Desired State* → *Reconcile Differences*. If a host node crashes and 2 Pods die, the controller detects the delta and orders the scheduler to spawn 2 replacement Pods on healthy nodes.

Real-World Cloud Example: Kubernetes Deployment controllers automatically restoring damaged microservice replicas on Amazon EC2 spot instances.

Related Concepts: Kube-controller-manager, etcd, Deployment, ReplicaSet.

3. The Kubernetes Control Plane Architecture

The Kubernetes Control Plane makes global decisions about the cluster (e.g., scheduling workloads, detecting node failures, and managing scaling). It consists of four core components communicating over secure TLS connections.

3.1 kube-apiserver: The Cluster Gateway

The **kube-apiserver** is the central front-end interface and communication hub of the entire Kubernetes cluster. It exposes a JSON/REST API over HTTPS that accepts commands from human administrators (via the `kubectl` CLI), worker node daemons, and internal controllers.

The kube-apiserver is completely stateless. It is the *only* component in the entire Kubernetes cluster permitted to communicate directly with the backend storage database (etcd). Every incoming request passes through a strict sequential pipeline:

1. **Authentication:** Verifies the caller's identity via X.509 client certificates, OpenID Connect (OIDC) tokens, or AWS IAM webhook authenticators.
2. **Authorization:** Evaluates permissions using Role-Based Access Control (RBAC) policies (e.g., verifying if user `alice` has permission to create Pods in namespace `production`).
3. **Dynamic Admission Control:** Executes Mutating Admission Webhooks (which can inject default labels or sidecar proxies) followed by Validating Admission Webhooks (which enforce security policies and reject non-compliant manifests).
4. **Persistence:** Serializes validated resource objects into JSON and writes them directly into etcd.

3.2 etcd: Distributed Consensus Storage

etcd is a strongly consistent, highly available, distributed key-value data store that serves as the single source of truth for all Kubernetes cluster state, configuration, and metadata.

etcd implements the **Raft consensus algorithm** to maintain strict data consistency across a multi-node cluster (typically 3 or 5 nodes in production). In Raft, a write is committed only when a majority (quorum) of etcd members acknowledge the transaction. etcd features a powerful *watch* API: controllers subscribe to specific key prefixes, and etcd immediately streams notifications when resources are added, modified, or deleted, eliminating the need for expensive database polling.

3.3 kube-scheduler: Workload Placement Engine

The **kube-scheduler** is responsible for watching for newly created Pods that have no assigned node and selecting the optimal worker node for each Pod to run on.

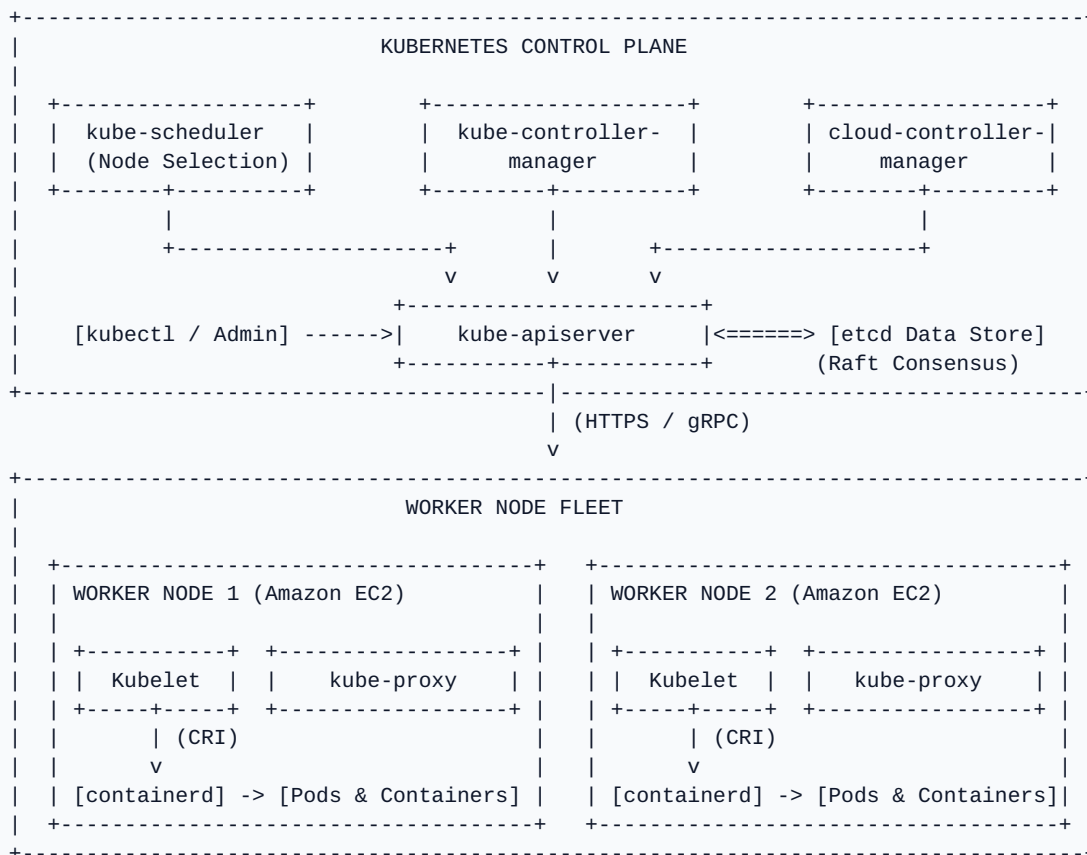
The scheduling cycle operates in two distinct phases for every unassigned Pod:

- **Filtering (Predicates):** The scheduler evaluates all nodes in the cluster and filters out those incapable of hosting the Pod. It checks available CPU/memory capacity against the Pod's resource requests, verifies node selector labels, checks Taints and Tolerations, and evaluates Node Affinity rules.
- **Scoring (Priorities):** The scheduler ranks the remaining eligible nodes using weighted scoring algorithms. It optimizes for even workload distribution (spreading Pods across different failure zones and Amazon EC2 instances) and data locality. The node with the highest cumulative score is selected, and the scheduler writes a *Binding* object to the kube-apiserver.

3.4 kube-controller-manager: The Brain of Reconciliation

The **kube-controller-manager** is a single binary that bundles multiple distinct controller loops into a single process. Each controller is an independent background loop that continuously reconciles a specific resource type:

- **Node Controller:** Monitors the health of worker nodes via periodic heartbeats. If a node stops responding for longer than the grace period (e.g., 40 seconds), the Node Controller marks the node as NotReady and triggers Pod eviction.
- **Deployment & ReplicaSet Controller:** Ensures that the exact number of Pod replicas declared in a Deployment specification are running across the cluster at all times.
- **Endpoints Controller:** Populates EndpointSlice objects, joining Kubernetes Services to the active IP addresses of healthy backend Pods.
- **ServiceAccount Controller:** Generates default ServiceAccounts and API access tokens for newly created namespaces.



4. Worker Node Architecture & Local Node Daemons

Worker nodes provide the compute, memory, storage, and networking capacity required to execute container workloads. Each worker node runs three fundamental software components.

4.1 Kubelet: The Node Execution Agent

The **Kubelet** is the primary node agent that runs on every worker node in the cluster. It communicates directly with the kube-apiserver and is responsible for ensuring that the containers described in PodSpecs are running and healthy on that specific physical or virtual machine.

The Kubelet does not create containers directly. Instead, it translates Kubernetes declarative specifications into operational instructions sent to the local container runtime (such as **containerd**) over a local Unix domain socket using the **Container Runtime Interface (CRI)** gRPC protocol. The Kubelet also monitors Pod health via Liveness, Readiness, and Startup probes, and periodically reports node status and resource utilization metrics back to the kube-apiserver.

4.2 kube-proxy: The Service Network Dispatcher

kube-proxy is a network agent that runs on every worker node, reflecting the networking services defined in the Kubernetes API. It enables network communication to Pods from inside or outside the cluster.

kube-proxy does not route traffic in user space in modern clusters. Instead, it programs low-level host packet filtering frameworks:

- **iptables Mode:** kube-proxy installs Destination NAT (DNAT) rules in the host kernel's iptables chains. When traffic hits a Service ClusterIP, iptables probabilistically rewrites the destination IP to one of the backend Pod IPs, performing Layer 4 load balancing directly inside the Linux kernel.
- **IPVS (IP Virtual Server) Mode:** In large clusters (over 1,000 services), iptables introduces sequential rule-matching latency. IPVS uses Linux kernel hash tables with $O(1)$ lookup complexity, supporting advanced load-balancing algorithms such as Least Connections and Round Robin.

4.3 Container Runtime (containerd): Local Execution

The worker node host runs a high-level container runtime, predominantly **containerd**. containerd receives CRI requests from the Kubelet to pull container images from registries (such as AWS ECR), set up root filesystems using Overlay2, and invoke **runc** to spawn containers within isolated Linux Namespaces and cgroups.

5. Workload Abstractions & Controllers

Kubernetes provides higher-level resource abstractions that define how Pods should be scaled, deployed, upgraded, and maintained across the cluster lifecycle.

Workload Primitive	Architectural Purpose	Lifecycle & Update Mechanism	Real-World Use Case
Pod	Atomic execution unit wrapping one or more containers.	Ephemeral; immutable; destroyed upon termination.	Single container or sidecar pattern.
ReplicaSet	Maintains a stable set of identical running Pod replicas.	Reconciles pod count using equality-based label selectors.	Low-level primitive managed by Deployments.
Deployment	Declarative updates for stateless applications.	Manages ReplicaSets; executes Rolling Updates and Rollbacks.	Stateless REST APIs, web applications, microservices.
StatefulSet	Manages stateful applications requiring unique identities.	Provides stable network IDs (pod-0, pod-1) and dedicated PersistentVolumes.	Distributed databases: PostgreSQL, MongoDB, Kafka, etcd.
DaemonSet	Ensures that all (or some) nodes run exactly one copy of a Pod.	Automatically places Pods on newly joined nodes.	Cluster log collectors (Fluentd), node monitoring (Prometheus Node Exporter).
Job & CronJob	Runs batch workloads to completion; scheduled execution.	Runs until specified successful completions occur.	Database migrations, batch report generation, backups.

6. Kubernetes Networking Model & Service Discovery

Kubernetes imposes a strict fundamental networking contract known as the **IP-per-Pod model**:

- Every Pod in the cluster receives its own unique, fully routable IP address within the cluster network.
- Every Pod can communicate with every other Pod on any node without using Network Address Translation (NAT).
- Agents on a node (such as the Kubelet) can communicate with all Pods on that same node.

6.1 Kubernetes Services & Service Types

Because Pods are ephemeral and receive dynamic IP addresses that change when Pods restart, clients cannot rely on static Pod IPs. A **Kubernetes Service** is an abstraction that defines a logical set of Pods and a consistent policy by which to access them:

- **ClusterIP (Default)**: Exposes the Service on an internal, cluster-private IP address. The Service is reachable only from within the cluster. Internal microservices communicate using ClusterIP.
- **NodePort**: Exposes the Service on each Node's IP at a static port (in the range 30000–32767). External traffic hitting <NodeIP> : <NodePort> is automatically routed to the backend Pods.
- **LoadBalancer**: Integrates with cloud providers (such as AWS) to automatically provision an external physical or virtual load balancer (such as an AWS Network Load Balancer or Application Load Balancer) that forwards traffic into the cluster.
- **Headless Service (clusterIP: None)**: Does not allocate a virtual ClusterIP. Instead, internal DNS returns direct A/AAAA records pointing to the individual IP addresses of the backend Pods, essential for StatefulSets and peer-to-peer database clustering.

7. Security, Admission Control & RBAC

Securing a Kubernetes cluster involves enforcing multi-layered authentication, authorization, and admission policies:

- **Role-Based Access Control (RBAC):** Regulates access to Kubernetes API resources based on the roles of individual users or ServiceAccounts. A Role defines namespace-scoped permissions (verbs: get, list, create on resources: pods, services), while a ClusterRole applies globally. RoleBindings bind identities to these roles.
- **Dynamic Admission Webhooks:** Intercepts API requests after authentication and authorization but before persistence into etcd. *Mutating Webhooks* can inject sidecar containers (e.g., Istio Envoy sidecars) or security contexts automatically. *Validating Webhooks* reject requests that violate governance policies (e.g., rejecting Pods running as root).
- **Pod Security Standards (PSS):** Replaces legacy PodSecurityPolicies with three built-in profiles enforced via admission controllers: *Privileged* (unrestricted), *Baseline* (prevents known privilege escalations), and *Restricted* (hardened, requiring rootless execution, dropping all Linux capabilities, and enforcing seccomp filters).

8. Practical Operational Troubleshooting

Diagnosing failures in Kubernetes requires isolating whether an issue is at the Pod level, node daemon level, or control plane level.

Failure State / Event	Root Cause	Diagnostic Commands	Remediation Action
CrashLoopBackOff	Application container starts, crashes immediately, and enters exponential restart backoff.	kubectl logs <pod> --previous kubectl describe pod <pod>	Fix application configuration, missing environment variables, or broken startup dependencies.
ImagePullBackOff / ErrImagePull	Kubelet failed to pull the specified container image from the registry.	kubectl describe pod <pod> (Inspect Events section)	Verify image name/tag spelling, registry network connectivity, and ECR IAM credentials/image pull secrets.
Pending (Scheduling Failed)	kube-scheduler cannot find an eligible worker node matching Pod requirements.	kubectl describe pod <pod>	Add worker nodes, reduce resource requests (CPU/RAM), or adjust NodeSelectors, Taints, and Tolerations.
Node NotReady	Kubelet stopped posting node heartbeats to kube-apiserver.	journalctl -u kubelet on node dmesg -T	Restart Kubelet daemon; resolve host disk pressure, PID exhaustion, or network partitioning.
OOMKilled (Exit Code 137)	Pod container exceeded its configured memory limit in the cgroup.	kubectl describe pod <pod> (Inspect Last State)	Increase resources.limits.memory or profile application for memory leaks.

9. Explicit Architectural Relationships & Dependencies

This section explicitly defines the structural relationships and interactions between Kubernetes components. These statements provide direct relational facts for Graph RAG entity linking and traversal:

- **kube-apiserver communicates with etcd:** The kube-apiserver is the only component that directly reads and writes cluster state data to the etcd key-value store.
- **kube-scheduler interacts with kube-apiserver:** The scheduler watches for unassigned Pods via the API server and writes node binding decisions back to the API server.
- **kube-controller-manager reconciles cluster state:** The controller manager continuously monitors actual state against desired state via the kube-apiserver and creates or destroys resources accordingly.

- **Kubelet communicates with kube-apiserver:** The Kubelet registers the node with the API server, watches for assigned Pods, and reports container execution health.
- **Kubelet communicates with containerd through CRI:** The Kubelet issues gRPC commands over a Unix domain socket to containerd to pull images and execute containers.
- **kube-proxy configures host iptables and IPVS:** kube-proxy reads Service and EndpointSlice definitions from the API server and programs Linux kernel packet filtering rules on worker nodes.
- **Deployments manage ReplicaSets:** A Kubernetes Deployment creates, scales, and manages underlying ReplicaSets to perform rolling updates and rollbacks.
- **ReplicaSets manage Pods:** A ReplicaSet ensures that a specified number of identical Pod replicas remain running across the cluster.
- **Services route traffic to Pods:** A Kubernetes Service uses label selectors to match active Pods and routes network traffic across their EndpointSlices.
- **Amazon EKS manages the Kubernetes Control Plane:** AWS manages and scales the kube-apiserver, etcd, and controller managers across multiple Availability Zones, while users run worker nodes on Amazon EC2.

10. Key Takeaways, Entities & Interview Questions

10.1 Key Takeaways

- Kubernetes is built on a declarative architecture governed by continuous reconciliation loops that drive actual state toward desired state.
- The Control Plane consists of kube-apiserver (API gateway), etcd (Raft state store), kube-scheduler (workload placement), and kube-controller-manager (state reconciliation).
- Worker nodes run the Kubelet (node execution agent), kube-proxy (network dispatcher), and containerd (container runtime).
- The Pod is the fundamental unit of deployment, grouping one or more containers that share a single network namespace and IP address.
- Kubernetes networking guarantees an IP-per-Pod model where all Pods communicate directly without NAT, while Services provide stable Layer 4 virtual IPs.

10.2 Important Entities & Classification

- **Control Plane Components:** kube-apiserver, etcd, kube-scheduler, kube-controller-manager, cloud-controller-manager.
- **Node Daemons:** Kubelet, kube-proxy, containerd, containerd-shim, runc.
- **Workload Resources:** Pod, ReplicaSet, Deployment, StatefulSet, DaemonSet, Job, CronJob.
- **Networking Resources:** Service, ClusterIP, NodePort, LoadBalancer, Headless Service, EndpointSlice, Ingress.
- **Security & Policy:** RBAC, Role, ClusterRole, RoleBinding, Mutating Webhook, Validating Webhook, Pod Security Standards (PSS).
- **Protocols & Standards:** Raft consensus, Container Runtime Interface (CRI), gRPC, HTTPS/TLS.
- **Cloud Integrations:** Amazon EKS, Amazon EC2, AWS IAM Webhook Authenticator, AWS Network Load Balancer.

10.3 Common Technical & Interview Questions

1. **What happens internally from the moment a user executes `kubectl apply -f deployment.yaml` until the container runs?**

Answer: `kubectl` sends an HTTPS POST request to kube-apiserver. The API server authenticates and authorizes the user, runs admission webhooks, and writes the Deployment object to etcd. The Deployment Controller detects the new Deployment and creates a ReplicaSet. The ReplicaSet Controller detects the ReplicaSet and creates Pods in a Pending state (with no node assigned). kube-scheduler filters and scores nodes, selecting an optimal node and binding the Pod to it. The Kubelet on that selected node detects the Pod assignment, calls containerd via CRI to pull images and set up namespaces, and starts the container processes.

2. Why is etcd the only component that communicates with kube-apiserver, and why is Raft consensus essential?

Answer: Restricting etcd access exclusively to kube-apiserver ensures strong data encapsulation, centralized schema validation, and unified security auditing. Raft consensus ensures that all state mutations require a quorum majority, preventing split-brain conditions and maintaining consistency during node network partitions.

3. How does kube-proxy enable Layer 4 load balancing without running a proxy process in user space?

Answer: kube-proxy configures the host Linux kernel's iptables or IPVS tables. When traffic is sent to a Service ClusterIP, the kernel intercepts the packet at Layer 4 and applies Destination NAT (DNAT), rewriting the destination IP directly to a healthy backend Pod IP with zero context switches to user space.

4. What is the operational difference between a Deployment and a StatefulSet?

Answer: Deployments manage stateless Pods that are interchangeable and receive random hashes in their names (e.g., web-67b4f-xyz). StatefulSets manage stateful workloads, providing deterministic, sequential network identities (e.g., db-0, db-1), ordered scaling/termination, and dedicated PersistentVolumeClaims that remain bound to specific Pod identities across restarts.

5. What causes a Pod to enter the CrashLoopBackOff state, and how should an engineer debug it?

Answer: CrashLoopBackOff indicates that the application container started, failed/exited with a non-zero exit code, and Kubernetes is waiting before restarting it again to prevent CPU thrashing. Debug using `kubectl logs <pod> --previous` to read the crash logs and `kubectl describe pod <pod>` to inspect exit codes and lifecycle events.